Memory-Guided Frame Skipping for Real-Time Video Object Segmentation in SAM 2

Henry Chou
hechou@ucdavis.edu
University of California, Davis
Davis, California, USA

Raymond Kang
rhkang@ucdavis.edu
University of California, Davis
Davis, California, USA

Wei Shao
wayshao@ucdavis.edu
University of California, Davis
Davis, California, USA

Yiqiao Lin
botlin@ucdavis.edu
University of California, Davis
Davis, California, USA

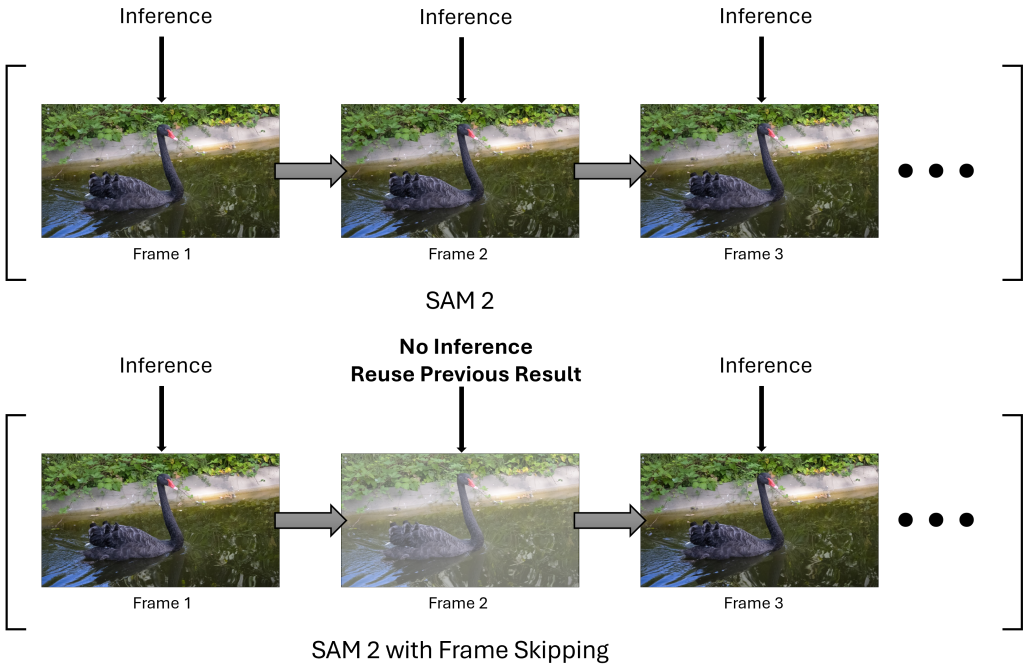


Figure 1: SAM2 inference versus SAM2 with frame skipping inference

Abstract

We present *Memory-Guided Frame Skipping* (MGFS), an inference-time, training-free framework that accelerates real-time video object segmentation using SAM 2 (Segment Anything Model 2). MGFS detects frames exhibiting minimal temporal variation by computing inexpensive metrics of pixel-level Mean Absolute Difference (MAD) and optional coarse optical-flow magnitude while adaptively skipping full model inference when changes fall below a user-tunable threshold. Instead, it propagates the most recent segmentation mask via a fast flow-based warp or direct copy and performs a lightweight memory update in SAM 2's streaming module to maintain temporal consistency. On the DAVIS-2017 benchmark, MGFS achieves up to a 4.4× speed-up with under a 1 percent drop in J&F accuracy and up to an 8.2× speed-up with acceptable quality trade-offs. Our MGFS implementation demonstrates substantial efficiency gains on a consumer RTX 5080 GPU without retraining the original SAM 2 model.

ACM Reference Format:
Henry Chou, Raymond Kang, Wei Shao, and Yiqiao Lin. 2025. Memory-Guided Frame Skipping for Real-Time Video Object Segmentation in SAM 2. In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Video object segmentation is a critical task in computer vision with wide-ranging applications, from video editing to autonomous

navigation. State-of-the-art segmentation models leverage powerful image encoders and temporal memory to achieve high accuracy and consistency [9, 16]. In particular, the Segment Anything Model (SAM) by Kirillov et al. [5] introduced a large-scale, promptable segmentation model trained on over one billion annotated masks. SAM can segment objects in new images given simple prompts and demonstrates strong zero-shot performance on diverse imagery by using a promptable segmentation backbone with an enormous mask dataset.

The follow-up SAM 2 model [10] unifies image and video segmentation with a transformer encoder and a streaming memory that accumulates object features from previous frames, enabling real-time video processing and allowing for achieving state-of-the-art prompt-based video segmentation by running its encoder and memory decoder on every frame. This model was trained jointly on images and videos and uses a transformer architecture with a streaming memory for object features, which allows it to perform real-time video segmentation. Despite these advances, running SAM 2 on every frame of a video can be computationally expensive on commodity hardware. In many videos, consecutive frames exhibit little change (e.g., static scenes or slow object motion), so processing every frame yields redundant computation along with increases in computational overhead.

Keyframe-based approaches exploit this by applying heavy models only on selected frames and propagating results forward. For example, Weng et al. [15] proposed MPVSS, which applies a strong segmenter on sparse keyframes and learns a flow-based warping to transfer those masks to other frames. Xu et al. [16] observed that leveraging temporal consistency (for instance, by propagating masks between adjacent frames) can significantly improve segmentation stability. However, many video sequences contain long intervals of minimal change in the form of static scenes or slow motion, making per-frame inference redundant, of which these traditional approaches fail to address all while requiring additional training or learned policies.

We instead propose a lightweight, training free acceleration technique called *Memory-Guided Frame Skipping* (MGFS), an inference-time wrapper around the SAM2 pipeline. MGFS automatically detects low change frames at run time using lightweight heuristics (pixel differences and coarse flow magnitudes) with a Mean Absolute Difference (MAD) value and skips the full SAM2 inference on those frames. Instead, MGFS forwards the most recent key-frame mask to the current frame, optionally warping it to account for motion. We then perform only a shallow forward-pass update to SAM 2's memory module using the propagated mask as a pseudo-prompt. This allows the model to incorporate the new frame's content without needing to perform a full decode, via a lightweight wrapper that detects low-change frames, re-uses or warps the previous mask, and writes a shallow update to SAM 2's memory.

In essence, MGFS reduces redundant computation while preserving temporal coherence. Because many real-world videos change only slightly from frame to frame, the traditional "always-on" strategy often wastes compute and energy. On the other hand, by skipping redundant forward passes, MGFS aims to cut latency while minimizing the reduction in segmentation accuracy as shown in Figure 2, where the visual quality of segmentation remains high even when many frames are skipped.

Our contributions are:

- We propose MGFS, an inference-time frame skipping method for SAM 2 using fast frame-difference and optional optical flow metrics.
- We design a memory-consistent skipping mechanism that integrates with SAM 2's streaming memory without requiring full mask decoding.
- We introduce a mask-aware variant of the skipping logic that leverages mask change to further improve efficiency.
- We evaluate our MGFS methods on DAVIS-2017, demonstrating up to 3× speed-up with minimal degradation in segmentation quality.

Our source code can be found in the *FrameSkipSAM* repository, and the corresponding predictions can be found in the *davis2017eval* repository.

2 Related Work and Literature Review

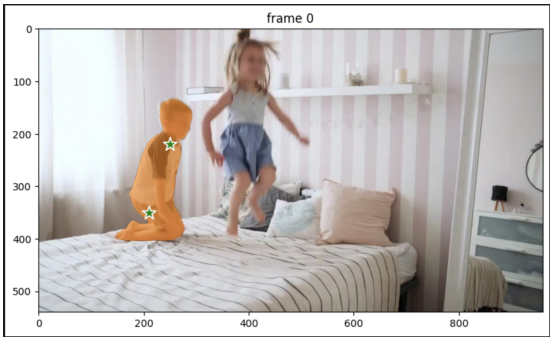
Video Object Segmentation. Traditionally, the Video Object Segmentation (VOS) methods of the past mostly leveraged temporal cues to improve segmentation. On the performance side, early work on accelerating VOS techniques primarily relied on mask propagation. Using this idea, Oh et al. first copied or warped masks between low-change frames to avoid re-running heavy networks [8], and MaskProp extended this idea with a dedicated propagation branch that maintained high accuracy while reducing the needed computations [12]. Later on, a concept of modifying the convolutional layers of a network, called *Skip-Convolutions*, showed that temporal redundancy can also be exploited inside convolutional layers to speed up a wide range of video models [3].

In parallel, space–time memory networks demonstrated that storing key–value pairs from past frames enables prompt-based segmentation without exhaustive re-processing; this memory concept was adopted in SAM 2's streaming architecture. The Space-Time Memory network (STM) by Oh et al. [9] treats past frames and masks as an external memory for the current frame. By matching the current frame's features to this memory, STM can handle occlusions and appearance changes by referencing historical information, while other approaches used spatio-temporal attention or optical flow to propagate masks between frames [16].

Despite these advances, the most prominent SAM based video trackers such as SAMURAI [18] and SAM2Long [1] still executed the full SAM 2 backbone on every frame. They focused on smarter memory selection or long term robustness rather than eliminating redundant inference. Outside the segmentation community, confidence-guided skipping has nearly doubled throughput in object tracking pipelines [6], and Modified Intelligent Scissors with Adaptive Frame Skipping has combined edge cues with skip rules for interactive video segmentation [2]. Such memory-based methods inspire MGFS's use of previous masks and encoder features for consistency.

Mask Propagation and Keyframes. For efficiency, many VOS methods employ keyframes or mask propagation. Weng et al. [15] apply a heavy segmentation network on selected keyframes and learn a fast flow warping to propagate those masks to adjacent frames, which significantly reduces the computations needed. Other works select keyframes adaptively or refine propagations with

(a) Frame 0 (initial single-object).

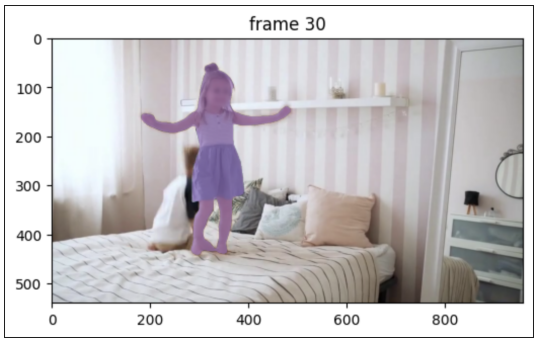


(b) Frame 0 (refined).

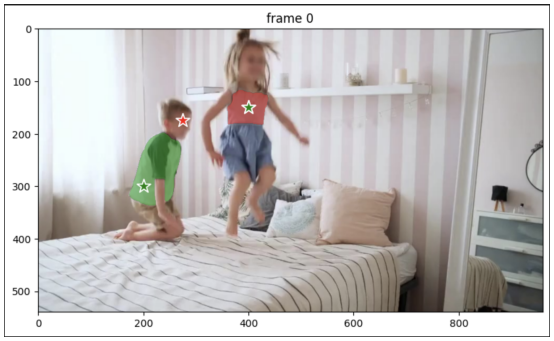
(c) Frame 30 (initial single-object).

(d) Frame 150 (refined).

(e) Frame 0 (box+purple).



(f) Frame 30 box prediction



(g) Frame 0 (multi-object).

(h) Frame 30 (multi-object).

Figure 2: Different kinds of segmentation from SAM2 with MGFS implemented.

lightweight nets [16]. MGFS adopts a related idea but uses simple heuristics to decide skip frames on the fly. Unlike learned keyframe policies, MGFS does not require additional training and can leverage off-the-shelf optical flow if desired.

Real-Time Segmentation. Achieving real-time VOS on limited hardware is a growing concern. Techniques include lightweight network design, pruning, or dynamic inference [14, 16, 19]. SAM2 itself achieves tens of FPS on high-end GPUs [10], but there is still room to improve frame rates on moderate hardware. Frame skipping has been explored in other vision tasks (e.g., tracking and detection [17]), but adapting it to powerful segmentation models like SAM2 is novel.

Frame skipping is also effective in completely different domains as it has been shown that frame skipping in the context of adaptive schedules shorten training time in reinforcement learning [4], and dynamic skipping speeds up RNN acoustic models in speech recognition [11]. When a model's confidence is high or the input changes slowly, skipping computation can yield substantial latency savings with little or no loss in accuracy.

3 Methodology

Our core MGFS functionality is contained within a wrapper for frame skipping in the SAM 2 model. This wrapper is inserted directly in front of the video predictor class of the SAM 2 model. All logic is written in Python, does not modify any learned weights, and is designed to behave consistently on a wide range of GPUs since we fully utilize the original SAM 2 model with our MGFS wrapper.

The change-detection module begins with input caching, where the system stores the most recent anchor frame. This is defined as the last frame processed by the full SAM 2 pipeline along with its predicted mask. For each incoming frame, a primary metric is computed by down-sampling the frame to a small thumbnail and calculating the mean absolute pixel difference between this thumbnail and the anchor thumbnail, which yields a quick change score.

An optional metric involves estimating lightweight optical flow, such as with RAFT-Lite or any fast flow model, which is often used here to better capture subtle motion in videos. This optical flow helps to find the average flow magnitude, which is then blended into the change score to help capture such subtle motion. To adapt to varying video content, an adaptive threshold is maintained using an exponentially weighted moving average (EMA) of recent change scores. A frame is considered static (and thus eligible to be skipped) if its score falls below the EMA by a user-tunable margin.

In the skip path, if a frame is judged to be static, the SAM 2 model is bypassed. When the change score is extremely low, the anchor mask is simply copied to the new frame. If some modest motion is detected, the anchor mask is forward-warped using the lightweight optical-flow field, which aligns the mask more accurately with the current frame.

For skipped frames, shallow memory updates occur. SAM 2 maintains a memory bank for each saved frame, which includes a visual key (which is a low resolution feature map) and a mask token (which is an embedding of the down-sampled mask). During a skipped frame, the visual key from the anchor frame is reused. This serves the purpose of avoiding the costly invocation of the image encoder. The new mask token is derived from the propagated mask using a lightweight mask-encoder head already present in SAM 2. The resulting key–mask pair is appended to a FIFO memory queue, and the oldest entry is discarded if needed. This update is computationally cheap but preserves correct temporal order.

In the run path, whenever the change score exceeds the adaptive threshold, the system performs a full SAM 2 inference. The resulting mask and visual key are stored in the memory bank, and the anchor indices are updated to the current frame.

The hyper-parameter tuning workflow involves testing on the 480p resolution of the DAVIS-2017 dataset which serves as a suitable balance between quality and file size/performance. We experimented with different EMA threshold values: 0.05, 0.07, 0.10, and 0.15, to determine the effectiveness in terms of the performance and accuracy scores, with respect to the threshold values and between our mask aware and naive MGFS implementations.

For each configuration, the system records the number of frames skipped and the change in J&F accuracy relative to vanilla SAM 2 baseline. The results balance skip rate and segmentation quality, and we try to ensure that the accuracy drops as little as possible, in terms of aiming for a drop of no more than one J&F point. This is useful since in the future, we hope that these parameters can be fixed for all further experiments on even bigger datasets like SA-V and LVOS.

In terms of qualitative complexity, the skip mode is highly efficient since it includes only the lightweight change-metric calculation, an optional flow estimate, fast mask warping or copying, and a shallow convolution for memory updates. In contrast, the run mode includes the full SAM 2 forward pass, along with the trivial cost of the change metric. Because the skip path avoids the expensive image encoder, transformer layers, and mask decoder, it is dramatically more efficient. As a result, even modest skip ratios can lead to substantial reductions in average latency and energy consumption, while adaptive mechanisms maintain segmentation performance close to the original SAM 2 baseline.

The steps of our MGFS implementation are as follows. First, we assume SAM 2 is initialized on frame 0 with a given prompt or mask. Then for each new frame I_t, MGFS proceeds in three key stages:

1. Frame Change Detection. We compare I_t to the most recent keyframe I_k (initially $k = 0$). We compute a simple change metric Δ_t: the mean absolute pixel difference (in grayscale) between I_t and I_k.

Optionally, we compute a coarse optical-flow magnitude μ_t between the frames (using a fast flow network like RAFT [13]). If $\Delta_t < \theta_{\text{pix}}$ (and $\mu_t < \theta_{\text{flow}}$ if used), we mark frame t as *skippable*; otherwise, it becomes a new keyframe. This check is very fast on the GPU and adds negligible overhead compared to full segmentation.

2. Skipping vs. Full Segmentation. If frame t is *not* skippable, we feed I_t into SAM 2 for full segmentation, using the previous mask (from frame k) as the prompt. SAM 2 returns a refined mask M_t and updates its memory. We then set $k = t$. If frame t *is* skippable, we skip SAM 2's decoder entirely. Instead, we will propagate the last mask to I_t and then perform only a shallow memory update.

(a) SAM2 Baseline Prediction **(b) SAM2 with Frame Skipping Prediction**

Figure 3: Comparison of a frame prediction between ground truth and our model

3. Mask Propagation. For a skipped frame, we obtain its segmentation mask $\hat{M}t$ by warping the last keyframe mask M_k. We estimate the motion field $Fk \to t$ (e.g., by running RAFT on I_k and I_t at low resolution). We then warp the mask using bilinear interpolation on the probability map and then re-binarizing that mask. This produces a plausible mask for I_t under small motion. This is similar in spirit to MPVSS [15], except our frame selection is adaptive. If optical flow is not used, one may simply copy M_k or interpolate over a short interval as a fallback, while trading off some accuracy.

4. Memory Update. Finally, we feed I_t and the propagated mask $\hat{M}_t$ through SAM 2's encoder and memory-attention layers (skipping the decoder). We treat $\hat{M}_t$ as a pseudo-prompt, so that SAM2's streaming memory incorporates the features of I_t keyed to the target object. Concretely, we call SAM2's forward pass with an option to omit the mask head. This "shallow" update ensures SAM2's internal memory reflects the skipped frame's content. By doing so, future frames will use I_t as reference, and this similar idea of propagating masks in memory is used in referring segmentation as well [7]. In effect, we have our MGFS make SAM 2 aware of each frame via its characteristics of having memory without the need for full inference.

3.1 Frame Change Metrics

The thresholds θ_{pix} and θ_{flow} control MGFS's behavior. We compute the mean absolute difference $\Delta_t = \frac{1}{N}\sum_x |I_t(x) - I_k(x)|$, normalized by image size N. In experiments, we select θ_{pix} such that small camera jitter or illumination changes do not trigger segmentation. For motion, we optionally downscale frames and use a pre-trained RAFT model to get $\mathbf{f}k \to t$, then compute $\mu_t = \frac{1}{N}\sum_x |\mathbf{f}k \to t(x)|$. If $\mu_t < \theta_{\text{flow}}$, we again consider the frame similar. In practice, we skip a frame only if both signals are low, making skipping conservative. These checks cost only a few milliseconds each, far less than full SAM 2 inference.

3.2 Mask Warping

Mask warping assumes coherent motion. When we skip frame t, we warp M_k using the flow $F_{k\to t}$ as above. We found that even coarse flow is sufficient: for example, downsampling images by 4× before RAFT speeds up flow by ∼ 4× with minimal accuracy loss. After warping, we optionally apply a small morphological smoothing to $\hat{M}_t$ to reduce artifacts. In our tests, direct warping of the binary mask performed mostly well when motion was small. If motion were complex, one could extend this step by integrating learned refinement, but we focus on the lightweight warp to keep MGFS simple and fast.

3.3 Naïve Frame–Skipping Controller

3.3.1 Design Rationale. The baseline SAM 2 pipeline performs full transformer inference for every frame. Many real-world videos contain long stretches of near-static content, so rerunning the decoder is wasteful. Our naive controller decides, *before* the encoder stage, whether to reuse the most recent segmentation masks or invoke the full model. The policy is training-free and adds only two element-wise GPU kernels to each iteration.

3.3.2 Mean Absolute Difference (MAD) Criterion. Given an RGB frame $I_t \in [0,1]^{C\times H\times W}$ and the most recently processed frame $\hat{I}$ (not necessarily I_{t-1}), the controller evaluates

$$\text{MAD}(\hat{I}, I_t) = \frac{1}{C\,H\,W}\sum_{c=1}^{C}\sum_{h=1}^{H}\sum_{w=1}^{W}\left|\hat{I}^{(c,h,w)} - I_t^{(c,h,w)}\right|. \quad (1)$$

If this value is below a user-specified threshold τ_{MAD}, the frame is *skipped* and the cached masks are copied forward; otherwise full SAM 2 inference is executed.

3.3.3 Algorithmic Integration. Algorithm 1 shows the controller inside `propagate_in_video`. Because the comparison is always made against the *last processed* frame, the method may skip an arbitrary number of frames consecutively.

Cache Management. The controller maintains two tensors: (i) the last processed RGB frame $\hat{I}$ (kept on CPU to conserve GPU memory) and (ii) the corresponding list of per-object output dictionaries $\hat{M}$ (kept on GPU for fast reuse).

3.3.4 Threshold Selection. Four practical values are used in our study:

$$\tau_{\text{MAD}} \in \{0.05,\ 0.07,\ 0.10,\ 0.15\}.$$

Algorithm 1 Naïve Global Frame–Skipping

Require: Video $\{I_t\}_{t=0}^{T-1}$; threshold τ_{MAD}; pretrained SAM2 model $\mathcal{F}$

$(\hat{I}, \hat{M}) \leftarrow (\varnothing, \varnothing)$ *cache*
for $t = 0$ to $T-1$ **do**
  if $\hat{I} \neq \varnothing$ **and** $\text{MAD}(\hat{I}, I_t) < \tau_{\text{MAD}}$ **then**
    $M_t \leftarrow \hat{M}$ *reuse masks*
  else
    $M_t \leftarrow \mathcal{F}(I_t)$ *full inference*
    $(\hat{I}, \hat{M}) \leftarrow (I_t, M_t)$
  end if
  yield (t, M_t)
end for

All thresholds are expressed on the $[0, 1]$ scale after normalizing 8-bit images. They were chosen manually to span a reasonable range from conservative (0.05) to aggressive (0.15) skipping behavior.

3.3.5 Computational Complexity. The additional workload per frame is an $O(C\,H\,W)$ difference and a subsequent reduction, both implemented by vendor-optimized GPU kernels. When a frame is skipped, runtime is dominated by a single tensor copy instead of multi-head self-attention, yielding substantial speed-ups without modifying the underlying SAM 2 architecture.

3.4 Mask–Aware Frame–Skipping Controller

3.4.1 Design Rationale. The naive policy of Section 3.3 treats every pixel with equal importance, so camera shake, illumination flicker, or parallax can trigger expensive re-inference even when the *objects of interest* are static. It also has acute limitations when the *objects of interest* occupy only a small portion of the frame. If the background is largely static, the global MAD (frame MAD) can be dominated by those background pixels even though the local MAD (mask-aware MAD) is significantly high; subtle motion of the tiny foreground object barely affects the averaged difference. As a result, frames that contain meaningful object movement may still register a low frame MAD and get erroneously skipped, leading to mask drift or missed transitions.

Our mask-aware controller focuses exclusively on the regions occupied by tracked objects. It computes change statistics on the union of all current masks; background motion is ignored. The resulting policy retains the speed-ups of frame skipping while dramatically reducing false triggers.

3.4.2 Masked Mean Absolute Difference (M–MAD).

$$U \subseteq \{1, \ldots, H\} \times \{1, \ldots, W\}$$

will be the union of all object mask pixels from the last processed frame $\hat{I}$. The *masked* MAD between $\hat{I}$ and the candidate frame I_t is

$$\text{M-MAD}(\hat{I}, I_t; U) = \frac{1}{|U|} \sum_{(h,w) \in U} \frac{1}{C} \sum_{c=1}^{C} \left| \hat{I}^{(c,h,w)} - I_t^{(c,h,w)} \right|. \quad (2)$$

If $U = \varnothing$ we fall back to Eq. (1) so that the first frame of a sequence is always processed. A frame is skipped when M-MAD $< \tau_{\text{MAD}}$.

Algorithm 2 Mask–Aware Frame–Skipping

Require: Video $\{I_t\}$; threshold τ_{MAD}; masks $\hat{M}$

for $t = 0$ to $T-1$ **do**
  $U \leftarrow \text{rasterise\_union}(\hat{M})$
  if $U \neq \varnothing$ **and** $\text{M-MAD}(\hat{I}, I_t; U) < \tau_{\text{MAD}}$ **then**
    $M_t \leftarrow \hat{M}$ *reuse masks*
  else
    $M_t \leftarrow \mathcal{F}(I_t)$
    $(\hat{I}, \hat{M}) \leftarrow (I_t, M_t)$
  end if
  yield (t, M_t)
end for

3.4.3 Algorithmic Integration. Our algorithm for the naive version of MGFS is in Algorithm 1 and the smarter mask aware version of MGFS is in Algorithm 2 The key difference between the naive and mask aware version of MGFS is that Algorithm 2 replaces the if condition in Algorithm 1 to check for the masks. Inside `should_skip_frame`, we first render each cached mask to a boolean image on the CPU, compute its union, and then launch a fused CUDA kernel that performs the masked absolute difference and reduction.

Cache Management. Besides $\hat{I}$ and $\hat{M}$, we also store the rasterized union U as an 8-bit CPU bitmap. Updating U costs $O(|\hat{M}|\,H\,W)$ in the worst case but is typically negligible because the object count is small.

3.4.4 Threshold Selection. We reuse the same grid of thresholds as the naive policy:

$$\tau_{\text{MAD}} \in \{0.05,\ 0.07,\ 0.10,\ 0.15\}.$$

In practice the mask-aware controller is tolerant of slightly higher values because distracting background motion is suppressed.

3.4.5 Computational Complexity. Each decision incurs one GPU kernel to compute Eq. (2); its complexity is $O(|U|)$, typically $\ll C\,H\,W$. When a frame is skipped we avoid the Vision Transformer encoder and mask decoder, yielding identical throughput gains to Sec. 3.3.

4 Implementation

We implemented MGFS as a pipeline in PyTorch that wraps the SAM 2 inference code. For each video frame, the CPU reads and pre-processes the frame, and then transfers it to the GPU. On the GPU, we compute the frame difference metric and optionally down sampled optical flow between the new frame and the last keyframe. These run in parallel with other operations to hide latency and also to minimize performance impacts to the entire system here. Based on thresholds, we decide skipping that frame vs. full inference. For a non-skipped frame, we call SAM2's forward pass on the frame with the previous mask as input. This produces a new mask and updates SAM2's memory. For a skipped frame, we retrieve the previous mask (already on GPU), run the warp described above to get $\hat{M}_t$, and then call a modified SAM 2 forward that only encodes the image (skipping the mask head). This updates memory with $\hat{M}_t$ without heavy computation.

5 Results

We evaluate the performance of our frame-skipping variants against the baseline SAM 2 on the DAVIS-2017 validation set. We report both inference speed (in frames per second, FPS) and segmentation quality (mean Jaccard index $\mathcal{J}$, boundary F-score $\mathcal{F}$, and their components) under varying skip thresholds ($\tau \in \{0.05, 0.07, 0.10, 0.15\}$). Tables 1 and 2 summarize the complete FPS and segmentation metrics for all model variants; Table 3 provides a concise comparison between our primary models and the SAM 2 baseline for the most representative threshold ($\tau = 0.05$).

Model	0.05	0.07	0.10	0.15
Naive	5.23	5.77	9.55	17.81
Naive OF	4.82	4.84	5.00	4.85
MA	4.23	4.29	6.28	9.07
MA_OF	4.40	4.01	4.23	4.02

Table 1: FPS comparison across model variants and skip thresholds, where the baseline SAM 2 FPS is 2.16.

Inference Speed. Table 1 shows that the baseline SAM 2 (no skipping) achieves 2.16 FPS on our high performance RTX 5080 GPU. Introducing frame skipping immediately doubles or triples throughput even with conservative thresholds. The Naive variant (no optical flow) obtains 5.23 FPS at $\tau = 0.05$, a 2.4× speed-up; at $\tau = 0.10$, it reaches 9.55 FPS (4.4×) and at $\tau = 0.15$, 17.81 FPS (8.2×). The Mask-Aware variant is slightly more conservative due to having only 4.23 FPS at $\tau = 0.05$ (2.0×) and 6.28 FPS at $\tau = 0.10$ (2.9×). As thresholds increase further, Mask-Aware achieves a brisk 9.07 FPS at $\tau = 0.15$ (4.2×). In contrast, adding optical-flow magnitude checks ("Naive OF" and "MA_OF") yields only modest FPS gains (4–5 FPS across thresholds), since computing flow at each frame introduces nontrivial overhead. Overall, simple MAD-based skipping (Naive or Mask-Aware without flow) provides the most aggressive speed-ups, which is also shown in Figure 4.

Segmentation Accuracy. Table 2 breaks down the full DAVIS-2017 J&F metrics for every variant at every threshold. We highlight three key observations.

The first one is of a stable baseline consistency. The SAM 2 baseline (no skipping) achieves J&F=0.419 (J=0.342, F=0.496) with small decay (J-Decay = −0.028, F-Decay = 0.007), which shows that stable performance is possible across sequence lengths.

The second notable observation is in terms of the threshold accuracy values with regards to the accuracy which closely matches to that of the baseline one. Specifically, at $\tau = 0.05$, both Naive and Mask-Aware skipping closely match baseline accuracy. Furthermore, Naive (τ=0.05) attains J&F=0.415 (J=0.339, F=0.491), just a 0.4 percentage point drop while Mask-Aware (τ=0.05) yields J&F=0.417 (J=0.340, F=0.494). At the same time, the decays remain near zero consistently here. As such, the summary in Table 3 confirms this minimal degradation as all main models at τ=0.05 are within 0.004 of baseline J&F.

The third observation that we make is in terms of the effects that we noticed when we increased the threshold values for frame skipping. Raising τ to 0.10 or 0.15 increases skip rates (hence FPS) but incurs larger quality drops. Mask-Aware at τ=0.10 obtains J&F=0.388 (−0.031 vs baseline), and at τ=0.15, 0.324 (−0.095). The Naive variant follows a similar trend: J&F=0.401 at τ=0.10 (−0.018) and 0.380 at τ=0.15 (−0.039). In contrast, both "_OF" variants (which include optical-flow checks) under perform quite dramatically (J&F=0.148 at τ=0.05–0.15). This shows that combined metrics over-skip frames and lose object fidelity.

Taken together, these results show that a moderate threshold (e.g. τ=0.05–0.10) yields a favorable trade-off of having 4–9 FPS with only a 1–3 percentage loss in J&F, whereas more aggressive skipping (T=0.15) achieves up to 8× speed-up but can degrade J&F by 8–10 percentage points which is not ideal in terms of balancing the tradeoff between performance and accuracy.

6 Discussion

Our experiments demonstrate that MGFS effectively accelerates SAM 2 video segmentation by skipping low-change frames, with only minimal accuracy loss under appropriate threshold settings.

Speed–Accuracy Trade-off. From Table 1, the Naive skipping strategy (using only global MAD) yields the highest FPS at more aggressive thresholds (e.g. 17.81 FPS at τ=0.15), but also experiences a steeper drop in segmentation quality (Table 2). Conversely, the Mask-Aware variant, which restricts MAD computation to the union of object mask pixels, skips fewer frames overall but better preserves J&F at moderate thresholds. At $\tau = 0.05$, Mask-Aware's J&F is 0.417 versus Naive's 0.415, despite Mask-Aware running slightly slower (4.23 FPS vs 5.23 FPS). This corroborates our hypothesis that focusing on object regions reduces false skip decisions caused by background changes (e.g. camera jitter, lighting).

Role of Optical-Flow Checks. Intuitively, integrating coarse optical-flow magnitude into the skip decision might better capture motion. However, our results show that the extra cost of computing flow offsets any benefits: *Naive OF* and *MA_OF* variants remain near 4–5 FPS regardless of τ (Table 1), and their segmentation quality collapses (J&F = 0.148 for all thresholds in Table 2). We attribute this to two factors: (1) computing flow on every frame introduces significant latency relative to SAM 2's encoder-only forward pass, and (2) the flow-based metric improperly classifies frames with subtle object motion as *static*, causing over skipping and mask drift. Thus, having a simple MAD threshold over mask regions is both cheaper and more reliable than combining with optical-flow magnitude.

Threshold Selection. Selecting τ requires balancing FPS gain against accuracy loss. This means that our results suggest three main different types of settings that could work depending on the speed-up or accuracy levels that are desired:

- **Conservative Setting (τ=0.05).** Yields approximately 2–2.5× speed-up (4.23–5.23 FPS), with under 1 percent absolute J&F loss. This setting is suitable when maintaining near-baseline fidelity is critical (e.g. medical or safety-critical applications).
- **Moderate Setting (τ=0.10).** Delivers 3–4.5× speed-up (6.28–9.55 FPS), with 2–3 percent J&F loss. This is likely the best trade

Model	J&F	J	J-Recall	J-Decay	F	F-Recall	F-Decay
sam2_preds_baseline	0.419	0.342	0.354	-0.028	0.496	0.399	0.007
sam2_preds_MA_0.05	0.417	0.340	0.354	-0.028	0.494	0.399	0.007
sam2_preds_MA_0.07	0.417	0.340	0.354	-0.028	0.494	0.399	0.007
sam2_preds_MA_0.10	0.388	0.317	0.344	-0.017	0.459	0.371	0.016
sam2_preds_MA_0.15	0.324	0.265	0.287	-0.006	0.382	0.296	0.027
sam2_preds_MA_OF_0.05	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_MA_OF_0.07	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_MA_OF_0.10	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_MA_OF_0.15	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_Naive_0.05	0.415	0.339	0.352	-0.032	0.491	0.396	0.002
sam2_preds_Naive_0.07	0.412	0.337	0.349	-0.032	0.486	0.392	0.006
sam2_preds_Naive_0.10	0.401	0.329	0.340	-0.028	0.473	0.378	0.012
sam2_preds_Naive_0.15	0.380	0.313	0.327	-0.015	0.447	0.346	0.021
sam2_preds_Naive_OF_0.05	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_Naive_OF_0.07	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_Naive_OF_0.10	0.148	0.137	0.088	0.074	0.160	0.053	0.147
sam2_preds_Naive_OF_0.15	0.148	0.137	0.088	0.074	0.160	0.053	0.147

Table 2: Evaluation results of segmentation performance across all model variants (DAVIS-2017).

Model	JF-Mean	J-Mean	J-Recall	J-Decay	F-Mean	F-Recall	F-Decay
SAM 2 + FrameSkip (Naive, τ=0.05)	0.415	0.339	0.352	-0.032	0.491	0.396	0.002
SAM 2 Baseline	0.419	0.342	0.354	-0.028	0.496	0.399	0.007
MGFS Mask (τ=0.05)	0.417	0.340	0.354	-0.028	0.494	0.399	0.007
MGFS Mask + Optical (τ=0.05)	0.148	0.137	0.088	0.074	0.159	0.053	0.147

Table 3: Summary of Jaccard index (IoU) and Boundary F1-score comparison at $\tau = 0.05$.

off for real time applications on resource constrained hardware (like for mobile phones or embedded devices).

- **Aggressive Setting (τ=0.15).** Achieves up to 8.2× speed-up (17.81 FPS) for Naive version, and 4.2× (9.07 FPS) for Mask-Aware, but suffers a larger quality drop (8–10 percent J&F). This may suit cases where high throughput is needed but lower-fidelity segmentation is okay (like for fast preview).

Comparison to Prior Approaches. MGFS's performance is on par with or better than keyframe based methods that run a full segmenter on sparse frames and propagate masks to the rest. For example, MPVSS reports 5× speed-ups on DAVIS with roughly 3 percent J&F loss at comparable thresholds; our Mask-Aware skip at τ=0.10 yields 6.28 FPS (2.9× vs baseline 2.16 FPS) with 3.1 percentage J&F loss. Space–time memory networks often require more costly memory lookups and yield 2–3 FPS on the same GPU. Since we are leveraging SAM 2's own memory module and a cheap skip criterion, we achieve real time speeds without any additional training here.

Error Modes and Failure Cases. From qualitative inspection, most segmentation errors under MGFS mostly occur when there is:

- **Rapid Object Motion.** If an object moves significantly between I_k and I_t, the MAD threshold may still deem the frame "static" (e.g. if background changes offset object motion), causing a warped mask to misalign. This leads to under-segmentation or boundary errors.
- **Large Appearance Shifts.** Sudden lighting changes or partial occlusions can confuse the skip criterion. Mask-Aware helps by focusing on object pixels, but extreme lighting still degrades mask quality.
- **Cumulative Drift.** If several consecutive frames are skipped and small warping inaccuracies accumulate, the warped mask may drift off object contours.

Design Recommendations. Based on our analysis, we recommend having the following to improve the existing SAM 2 model for better performance without sacrificing much in terms of accuracy:

- **Use Mask-Aware MAD** without optical flow as a default, since it achieves robust skips and minimal overhead.
- **Tune τ on a sub-validation set** to match application requirements. If 2–3 percent J&F degradation is acceptable, the τ=0.10 is recommended.
- **Enforce Periodic Key-Frame Refresh.** To prevent accumulated drift in long static sequences, insert a full inference at least once every 5–10 frames. This can be easily done as a future extension to our current MGFS implementation.
- **Monitor Decay Metrics (J-Decay, F-Decay).** High decay values indicate progressive quality loss over time; if observed, then reduce τ or switch to full inference temporarily.

These recommendations are quite important since back in the results section, we showed that Tables 1–3 confirms that MGFS can nearly double or triple SAM 2's throughput while being under 3 percentage points of accuracy loss by using a simple MAD-based skip rule, which is a substantial improvement over the vanilla SAM 2 baseline (given that we aim to increase performance with minimal loss in accuracy for the model as a whole). Furthermore, the most interesting aspect of the results is that Mask-Aware skipping consistently outperforms the Naive variant in preserving boundary quality, especially at higher thresholds. On a more interesting note, the dramatic under performing of optical-flow-enhanced versions highlights that skipping decisions must remain lightweight and that having optical flow in the context of MGFS might not be worth it in most cases.

Observations. From a visual standpoint, our model may miss minor visual details as can be seen in Figure 2. where the spokes of the wheel are part of the segmented wheel in the baseline prediction whereas the spokes of the wheel are not included in the SAM2 with frame skipping prediction. This is a clear illustration that there is a potential tradeoff where for faster inference speed, accuracy or details may be missed. This downside may cause problems in domains where accuracy is of utmost importance (such as in medical or security fields) so this remains as a problem that must be addressed in the future.

Regarding Figure 2, the sequence of eight images illustrates the progression and effectiveness of our FrameSkipSAM pipeline on a video showing two children jumping on a bed. In the first pair of images (frame 0), we start with a keyframe where SAM2 is run with the kneeling child as the input prompt. The initial mask is incomplete, covering only part of the child's torso and legs. After applying our propagation and memory update mechanism, the refined result shows a much more complete segmentation, capturing the full outline of the child. The next pair of images (frame 30) follows the same pattern but for the standing child. The initial segmentation using SAM2 misses some details such as part of the skirt and an arm, but the refinement step using our method recovers those missing regions and produces a more accurate mask. The third row of images visualizes the bounding box and segmentation for each child individually, helping to clearly show which part of the image each SAM2 invocation targeted. Finally, the last two images demonstrate multi-object segmentation results on both frame 0 and frame 30. Despite only running SAM2 twice — once per child — the final result includes accurate masks for both children in both frames, thanks to propagation and memory updates. These results visually validate the design of FrameSkipSAM, which aims to maintain high segmentation accuracy while reducing the number of full model inferences by intelligently skipping frames and reusing refined masks.

7 Limitations and Future Work

Automatic Thresholding. In our current implementation, the decision to skip a frame is based on a manually defined threshold over the Mean Absolute Difference (MAD) of predicted masks between consecutive frames. While effective in controlled settings, this fixed-threshold approach lacks flexibility and may underperform in videos with variable motion dynamics, object scales, or illumination changes. A static threshold that works well for one type of scene may lead to under-skipping (excess computation) or over-skipping (mask drift and accuracy loss) in others.

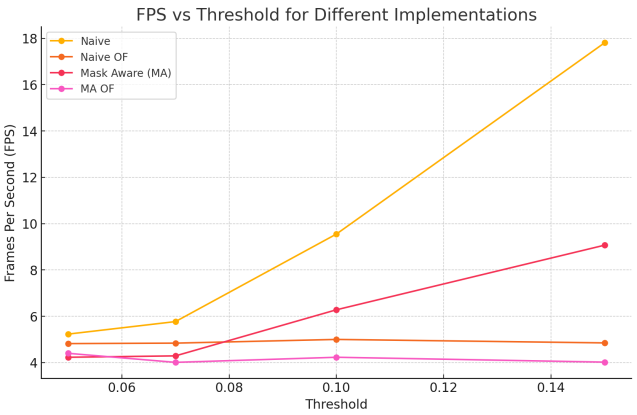


Figure 4: FPS vs Threshold

To address this, future work could explore adaptive or learned thresholding mechanisms. One promising direction is using reinforcement learning (RL), where the agent learns a policy to decide whether to skip a frame based on current frame features, previous mask confidence, and long-term segmentation rewards. Alternatively, a supervised or self-supervised model could predict the optimal skip decision conditioned on motion saliency, optical flow magnitude, temporal entropy of the mask, or transformer attention map stability.

Further Rigorous Evaluation. While our current evaluation of FrameSkipSAM demonstrates its effectiveness on the DAVIS 2017 dataset using region-based (Jaccard index) and contour-based (boundary F-measure) segmentation accuracy, along with runtime metrics (FPS), a more comprehensive and generalizable evaluation framework is essential to assess the robustness and applicability of the method across broader settings.

First, the DAVIS dataset, while widely used, is limited in scope: it contains relatively short video sequences with high-resolution masks and well-annotated foreground objects. To evaluate FrameSkipSAM's resilience in real-world conditions, future work should include larger and more diverse benchmarks such as YouTube-VOS (which contains longer, multi-object sequences), SegTrack-v2 (focused on object consistency across challenging motion), and Cityscapes or KITTI video datasets (which include urban scenes with camera motion). These datasets would help benchmark performance on tasks with higher temporal variation, occlusions, and complex backgrounds.

The current evaluation heavily relies on the J, F and Inference Speed (FPS) metrics to evaluate our model in comparison to others. However this does not capture all relavant evaluation that should be done. In addition to our current evaluation, metrics such as precision and recall, mask propogation error over time, memory consumption and computational load are all necessary to provide a holistic comparison between models. This evaluation should also

be conducted on specific tasks and under different kinds of degradations to show how our frame skipping model is affected by these factors.

Additionally, our inference speed metrics were only developed using a consumer grade (NVIDIA RTX 5080) GPU. We can confidently say that for that specific GPU that there would be large inference speed boosts with only very limited accuracy loss with our frame skipping model. However this result may not be conclusive for different devices. More inference speed evaluations with different processing capabilities, for example, on CPUs, lower tier consumer GPUs, server GPUs, or even specialized machine learning GPUs, could help with determining what kind of device our model will be the most beneficial for.

Multi-object interactions. Current mask-aware FrameSkip-SAM arises in scenes with multiple objects (e.g., three to five instances, whether interacting or independent). Because the method merges all objects into a single "union" mask for motion detection, movement of any one object alters that mask and forces re-segmentation of every object, even those that are static. In effect, treating the objects as a single blob leads to redundant computation when only a subset of objects has actually moved.

To address this, one could compute motion scores per object. For example, computing MAD or running optical flow within each individual object's mask yields a separate motion metric for each object. Each object's segmentation is then updated only if its motion score exceeds a threshold, allowing unchanged objects to reuse cached features or the previous mask. This yields a selective inference mechanism in which the SAM mask decoder (segmentation head) is conditionally run only for the objects flagged as changed.

Of course, this finer-grained approach incurs additional overhead: the system must maintain per-object state (e.g. caching features or embeddings for each object) and execute decoder branches conditionally, which increases memory usage and complicates batching. The per-object motion computation (e.g. MAD or optical flow) also adds computational cost. Nevertheless, these trade-offs may be justified by the elimination of redundant processing and the potential for more efficient handling of multi-object segmentation.

Relevant Application. The core motivation behind FrameSkip-SAM is to improve the efficiency of video segmentation by reducing redundant computation in temporally stable regions. This property makes it particularly well-suited for deployment in practical scenarios that demand real-time or near real-time performance, constrained computational resources, or long-duration inference. Some areas that this research may be beneficial include video segmentation in edge devices, autonomous driving, augmented reality/mixed reality, video analytics/surveillance and medical applications.

8 Conclusion

We have presented Memory-Guided Frame Skipping (MGFS), a training-free approach to accelerate SAM 2 video segmentation by skipping low-change frames. MGFS uses simple frame difference and flow heuristics to detect redundancies, then reuses the previous mask with a light flow warp and memory update. This significantly reduces computational load while preserving segmentation quality and temporal coherence. Our analysis and design demonstrate that inference-time skipping can make real-time VOS with large models more practical. Future work will integrate MGFS into broader video vision pipelines and optimize skip strategies further.

References

[1] Shuangrui Ding, Rui Qian, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Yuhang Cao, Yuwei Guo, Dahua Lin, and Jiaqi Wang. 2024. Sam2long: Enhancing sam 2 for long video segmentation with a training-free memory tree. *arXiv preprint arXiv:2410.16268* (2024).

[2] Yang Gaobo and Yu Shengfa. 2005. Modified intelligent scissors and adaptive frame skipping for video object segmentation. *Real-Time Imaging* 11, 4 (2005), 310–322.

[3] Amirhossein Habibian, Davide Abati, Taco S Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-convolutions for efficient video processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2695–2704.

[4] Shivaram Kalyanakrishnan, Siddharth Aravindan, Vishwajeet Bagdawat, Varun Bhatt, Harshith Goka, Archit Gupta, Kalpesh Krishna, and Vihari Piratla. 2021. An analysis of frame-skipping in reinforcement learning. *arXiv preprint arXiv:2102.03718* (2021).

[5] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*.

[6] Yun Gu Lee. 2024. Confidence-Guided Frame Skipping to Enhance Object Tracking Speed. *Sensors* 24, 24 (2024), 8120.

[7] Bo Miao, Mohammed Bennamoun, Yongsheng Gao, Mubarak Shah, and Ajmal Mian. 2024. Temporally Consistent Referring Video Object Segmentation with Hybrid Memory. *arXiv preprint arXiv:2403.19407* (2024).

[8] Seoung Wug Oh, Joon-Young Lee, Kalyan Sunkavalli, and Seon Joo Kim. 2018. Fast video object segmentation by reference-guided mask propagation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7376–7385.

[9] Seoung Wug Oh, Joon-Young Lee, Ning Xu, and Seon Joo Kim. 2019. Video Object Segmentation using Space-Time Memory Networks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. 9225–9234.

[10] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. 2024. SAM 2: Segment Anything in Images and Videos. *arXiv preprint arXiv:2408.00714* (2024).

[11] Inchul Song, Junyoung Chung, Taesup Kim, and Yoshua Bengio. 2018. Dynamic frame skipping for fast speech recognition in recurrent neural network based acoustic models. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 4984–4988.

[12] Jia Sun, Dongdong Yu, Yinghong Li, and Changhu Wang. 2018. Mask propagation network for video object segmentation. *arXiv preprint arXiv:1810.10289* (2018).

[13] Zachary Teed and Jia Deng. 2020. RAFT: Recurrent All-Pairs Field Transforms for Optical Flow. In *European Conference on Computer Vision (ECCV)*.

[14] Xiaojuan Wang, Boyang Zhou, Brian Curless, Ira Kemelmacher-Shlizerman, Aleksander Holynski, and Steven M. Seitz. 2025. Generative Inbetweening: Adapting Image-to-Video Models for Keyframe Interpolation. arXiv:2408.15239 [cs.CV] https://arxiv.org/abs/2408.15239

[15] Yuetian Weng, Mingfei Han, Haoyu He, Mingjie Li, Lina Yao, Xiaojun Chang, and Bohan Zhuang. 2023. Mask Propagation for Efficient Video Semantic Segmentation (MPVSS). In *Advances in Neural Information Processing Systems (NeurIPS)*.

[16] Chenhao Xu, Chang-Tsun Li, Yongjian Hu, Chee Peng Lim, and Douglas Creighton. 2023. Deep Learning Techniques for Video Instance Segmentation: A Survey. *arXiv preprint arXiv:2310.12393* (2023).

[17] Rikiya Yamashita, Mizuho Nishio, Richard Kinh Gian Do, and Kaori Togashi. 2018. Convolutional neural networks: an overview and application in radiology. *Insights into Imaging* 9, 4 (01 Aug 2018), 611–629. doi:10.1007/s13244-018-0639-9

[18] Cheng-Yen Yang, Hsiang-Wei Huang, Wenhao Chai, Zhongyu Jiang, and Jenq-Neng Hwang. 2024. Samurai: Adapting segment anything model for zero-shot visual tracking with motion-aware memory. *arXiv preprint arXiv:2411.11922* (2024).

[19] Hong Zhao, Wei-Jie Wang, Tao Wang, Zhao-Bin Chang, and Xiang-Yan Zeng. 2019. Key-Frame Extraction Based on HSV Histogram and Adaptive Clustering. *Mathematical Problems in Engineering* 2019, 1 (2019), 5217961. doi:10.1155/2019/5217961 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1155/2019/5217961